

claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\workflows\wf_3d536535-241\agent-  
a1dc462074712a03b.jsonl`  
- SHA-256: `6eccdaff2eecef9809c8083059f487c3df7be3e1a5a832683a5341c68695ea99`  
- Source modified: `2026-06-16T03:23:37+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_3d536535-241`  
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`
```

Transcript

1. user (2026-06-16T03:22:18.703Z)

Adversarial Claim Verifier (voter 3/3)

Be SKEPTICAL. Try to REFUTE this claim. ?2/3 refutations kill it.

Research question

Research question: ****What concrete, recent (2022?2025) techniques and evaluation practices should a local, commercially-licensed badminton rally-detector adopt to close the quality gap with a strong cloud VLM (Gemini), given a tiny labeled set (~6 ? ~16 golden videos)?**** Produce an adversarially-verified, cited report organized around the four dimensions below. Prefer specific named methods, papers (arXiv/venue), and reported numbers over generic advice; flag anything you cannot verify.

CONTEXT (so you target the real gap, not generic ML):

- System: a local pipeline detects the shuttle per-frame (WASB / TrackNet, MIT-licensed) ? a heuristic turns the trajectory into candidate rally windows ? today an optional Gemini "handoff" confirms rallies. We just measured a fully-local (no-AI) run vs Gemini on 3 matches: local emits far FEWER but MUCH LONGER windows (mean 65s vs 6.1s; one window spanned a whole 174s clip) ? it does NOT miss activity, it FAILS TO SEGMENT. Root cause (code-confirmed): a frame is "active" iff the shuttle is visible AND moving above a small per-second-normalized velocity threshold; active frames within a 2 s gap are merged; there is NO max-window cap and NO inactivity/boundary split. So "shuttle is moving" is conflated with "rally in progress," and dead-time + serves + knock-ups bridge rallies into mega-windows. Shuttle tracking itself is excellent (96?99% per-frame visibility).

- HARD CONSTRAINTS: weights/data/code must be MIT/Apache-2.0/BSD only (no viral/NC/custom licenses, reject Gemma-3/Llama-4-MAU-cap); we may NEVER train owned weights on paid-LLM (Gemini/OpenAI/Anthropic) outputs (terms/copyright), though paid LLMs may be used at inference; minors excluded from any training pool; runs on a single local GPU.

- WHAT WE ALREADY HAVE (do NOT re-survey the basics ? go deeper/newer): the architecture skeleton is known to be standard Temporal Action Localization/Segmentation (proposal+scorer) + late/score-level fusion + stacked generalization; in-domain prior art (MonoTrack, ShuttleSet, See et al. 2024/25 non-broadcast badminton rally start/end, Ghosh et al. point-segmentation, tennis/TT play-break state machines) is already cited. We have a working eval harness: leave-one-video-out (grouped by match), temporal-IoU F1 @0.5, F1@{0.1,0.25,0.5}, mAP@tIoU, segment-count-ratio, bootstrap 95% CIs, paired exact sign-test, Platt/isotonic calibration + Brier/ECE, a tiny numpy LogisticRegression scorer over per-window feature columns, an experiment leaderboard, and an "eval?serve" guard (a cue measured +0.074 F1 on permissive proposal windowing but ?0.008 on the coarser served windowing ? reverted). Available but default-OFF per-window signals: net-crossing count (rally gate), 5 person/court features, 3 audio-hit features.

THE FOUR DIMENSIONS:

1) WINDOWING & BOUNDARIES (Temporal Action Localization/Segmentation): beyond proposal+score ? what are the best ***recent*** methods for (a) turning a dense per-frame signal into well-segmented

actions, (b) explicitly controlling OVER-segmentation/merging, (c) boundary detection/regression and splitting merged segments? Cover e.g. MS-TCN/MS-TCN++, ASRF (action-boundary regression), BMN/BSN boundary-matching, ActionFormer/TriDet/TemporalMaxer (actionness + boundary heads), smoothing/boundary losses, and the standard metrics for over-segmentation (segmental F1@k, edit/Levenshtein score, mAP@tIoU). Which transfer to a 1-D shuttle-trajectory + auxiliary-cue setting on tiny data?

2) SHUTTLE-TRAJECTORY ? RALLY SIGNAL PROCESSING (racket-sports specific): how do published systems convert ball/shuttle tracking into rally/point segments and distinguish "in-play" from dead-time? Cover rally-state machines, serve/let detection, net-crossing & two-sided-occupancy cues, hit/contact (audio + kinematic) detection, and fps-invariant trajectory features. What features best separate "rally" from "shuttle merely moving"?

3) SMALL-DATA STRATEGIES: with ~6?16 labeled videos, what gives the most quality per label ? active learning (which videos/segments to label next; uncertainty/diversity/core-set), semi-/weakly-/point-supervised TAL (e.g. SF-Net, timestamp supervision), self-training/pseudo-labeling, and data-efficient TAL? Separately: the legally-clean ways to exploit a strong teacher (Gemini) at INFERENCE for weak/pseudo-labels or as an evaluation oracle WITHOUT its outputs becoming training data for owned weights ? what does the literature say about teacher-student / distillation boundaries and label-noise handling, and what governance distinctions matter?

4) EVALUATION METHODOLOGY & RIGOR at small n: trustworthy measurement with ~6?16 videos ? bootstrap/permutation tests, paired sign-tests, leave-one-group-out vs RL00CV bias, calibration (reliability, ECE), the over-segmentation-blindness of plain tIoU, shadow/A-B evaluation, regression gates, and concrete pitfalls (label noise, annotator agreement/IAA, multiple-comparison/over-fitting traps, eval-vs-serve drift). What metrics + protocols should a small-data sports-segmentation leaderboard standardize on?

Deliverable: a structured, cited report with, per dimension, the 3?5 highest-leverage techniques that APPLY to our constrained setting (small data, local, license-clean, 1-D trajectory + cues + learned scorer), each with: what it is, why it fits our over-merge/small-data problem, the expected difficulty/payoff, and the citation. Call out which ideas are low-effort high-confidence wins vs research bets. Be explicit about anything unverified or license-incompatible.

Claim under review

"On THUMOS14, TriDet reaches 69.3% average mAP, beating the prior best (ActionFormer at 66.8%) by 2.5 points while using only 74.6% of its latency, showing a convolution-only (no self-attention) detector can outperform transformer-based TAD at lower cost."

****Source:**** https://openaccess.thecvf.com/content/CVPR2023/papers/Shi_TriDet_Temporal_Action_Detection_With_Relative_Boundary_Modeling_CVPR_2023_paper.pdf (primary)

****Supporting quote:**** "TriDet hits an average mAP of 69.3% on THUMOS14, outperforming the previous best by 2.5%, but with only 74.6% of its latency."

Checklist

1. Is the claim actually supported by the quote, or is it an overreach/misread?
2. WebSearch for contradicting evidence ? does any credible source dispute or heavily qualify this?
3. Is the source quality sufficient for the claim's strength? (extraordinary claims need primary sources)
4. Is the claim outdated? (check dates ? old claims about fast-moving fields are suspect)
5. Is this a marketing claim / press release / cherry-picked benchmark / forum speculation?

****refuted=true**** if: unsupported by quote / contradicted / low-quality source for strong claim / outdated / marketing fluff.

****refuted=false**** ONLY if: claim is well-supported, current, and source quality matches claim strength.

Default to refuted=true if uncertain.

Structured output only. Evidence MUST be specific.

2. assistant (2026-06-16T03:22:24.707Z)

I'll verify this claim about TriDet's performance on THUMOS14 by examining the primary source.

3. assistant (2026-06-16T03:22:27.987Z)

I'll fetch the primary source PDF to verify the specific numbers in the claim.

4. user (2026-06-16T03:22:43.437Z)

The server returned HTTP 403 Forbidden.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

5. assistant (2026-06-16T03:22:45.656Z)

The CVF page is blocking direct fetch. Let me try the arXiv version and search for corroborating details.

6. user (2026-06-16T03:22:52.618Z)

TriDet THUMOS14 Results - Exact Numbers

****1. TriDet's average mAP on THUMOS14:****
69.3%

****2. Previous best method and its mAP:****
The abstract does not name the specific previous method. It only states TriDet outperforms "the previous best" without attribution.

****3. Margin of improvement:****
2.5 percentage points

****4. Latency comparison:****
TriDet achieves this performance "with only 74.6% of its latency" compared to the previous best method.

****5. Self-attention and SGP layer:****
TriDet uses self-attention. The abstract states the SGP (Scalable-Granularity Perception) layer is designed to "mitigate the rank loss problem of self-attention that takes place in the video features and aggregate information across different temporal granularities."

****Relevant abstract quote:****
"TriDet hits an average mAP of 69.3% on THUMOS14, outperforming the previous best by 2.5%, but with only 74.6% of its latency."

7. user (2026-06-16T03:22:53.385Z)

Web search results for query: "TriDet Temporal Action Detection Relative Boundary Modeling THUMOS14 average mAP 69.3 ActionFormer latency"

Links: [{"title": "TriDet: Temporal Action Detection with Relative Boundary Modeling | OpenReview", "url": "https://openreview.net/forum?id=3C1Gs6vop9"}, {"title": "[2303.07347] TriDet: Temporal Action Detection with Relative Boundary Modeling", "url": "https://arxiv.org/abs/2303.07347"}, {"title": "TriDet: Temporal Action Detection with Relative Boundary Modeling", "url": "https://arxiv.org/pdf/2303.07347"}, {"title": "TriDet: Temporal Action Detection", "url": "https://www.emergentmind.com/papers/2303.07347"}, {"title": "TriDet: Temporal Action Detection with Relative Boundary Modeling - ADS", "url": "https://ui.adsabs.harvard.edu/abs/2023arXiv230307347S/abstract"}, {"title": "Paper page - TriDet: Temporal Action Detection with Relative Boundary Modeling", "url": "https://paperswithcode.com/paper/tridet-temporal-action-detection-with"}, {"title": "[Quick Review] TriDet: Temporal Action Detection with Relative Boundary Modeling", "url": "https://liner.com/review/tridet-temporal-action-detection-with-relative-boundary-modeling"}, {"title": "TriDet: Temporal Action Detection with Relative Boundary Modeling Dingfeng Shi?", "url": "https://openaccess.thecvf.com/content/CVPR2023/papers/Shi_TriDet_Temporal_"}]

```
Action_Detection_With_Relative_Boundary_Modeling_CVPR_2023_paper.pdf"}, {"title": "GitHub - dingfengshi/TriDet: [CVPR2023] Code for the paper, TriDet: Temporal Action Detection with Relative Boundary Modeling . GitHub", "url": "https://github.com/dingfengshi/TriDet"}, {"title": "CVPR Poster TriDet: Temporal Action Detection With Relative Boundary Modeling", "url": "https://cvpr.thecvf.com/virtual/2023/poster/21956"}]
```

Based on the search results, here are the key findings about TriDet:

TriDet hits an average mAP of 69.3% on THUMOS14, outperforming the previous best by 2.5%, but with only 74.6% of its latency.

The previous best method being referenced appears to be ActionFormer, as TriDet's code is built upon the codebase from ActionFormer and Detectron2.

TriDet is a one-stage framework for temporal action detection that addresses the problem of imprecise boundary predictions by proposing a novel Trident-head to model the action boundary via an estimated relative probability distribution around the boundary.

TriDet achieves state-of-the-art performance on three challenging benchmarks: THUMOS14, HACS and EPIC-KITCHEN 100, with lower computational costs, compared to previous methods.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

8. assistant (2026-06-16T03:22:57.767Z)

The abstract from arXiv is confirmed verbatim. Now let me critically examine the two contested parts of the claim: (1) the "ActionFormer at 66.8%" baseline number, and (2) the "convolution-only (no self-attention)" characterization. The WebFetch result above flagged that SGP is designed to MITIGATE the rank-loss problem OF self-attention, which suggests SGP may REPLACE self-attention rather than the model "using" it. Let me dig into both.

9. user (2026-06-16T03:23:04.156Z)

Analysis of TriDet Performance Claims

ActionFormer Baseline: The document does not specify ActionFormer's THUMOS14 mAP score. It only states that TriDet surpasses "the previous best" by 2.5%.

SGP Layer and Self-Attention: TriDet replaces self-attention with convolution-based architecture. The relevant quote is:

"By replacing self-attention with a convolutional-based architecture that includes an instant-level branch and a window-level branch, the SGP layer enhances feature discrimination and reduces computational overhead."

Performance Metrics:

- **69.3% mAP:** TriDet achieves "an average mAP of 69.3% on THUMOS14"
- **2.5% improvement:** It outperforms "the previous best by 2.5%"
- **74.6% latency:** TriDet operates "with only 74.6% of its latency" relative to prior methods

The document does not identify which specific method held the previous best result.

10. user (2026-06-16T03:23:05.789Z)

Web search results for query: "TriDet SGP layer replaces self-attention "without" attention convolution ActionFormer 66.8 THUMOS14 average mAP"

Links: [{"title": "TriDet: Temporal Action Detection with Relative Boundary Modeling", "url": "https://arxiv.org/pdf/2303.07347"}, {"title": "TriDet: Temporal Action Detection", "url": "https://www.emergentmind.com/papers/2303.07347"}, {"title": "Noname manuscript No. (will be inserted by the

editor)","url":"https://arxiv.org/pdf/2309.05590"},"title":"[2303.07347] TriDet: Temporal Action Detection with Relative Boundary Modeling","url":"https://arxiv.org/abs/2303.07347"},"title":"TriDet: Temporal Action Detection with Relative Boundary Modeling | OpenReview","url":"https://openreview.net/forum?id=3C1Gs6vop9"},"title":"Benchmarking Data Efficiency and Computational Efficiency","url":"https://cveu.github.io/2023/papers/11.pdf"},"title":"TriDet: Temporal Action Detection with Relative Boundary Modeling Dingfeng Shi?","url":"https://openaccess.thecvf.com/content/CVPR2023/papers/Shi_TriDet_Temporal_Action_Detection_With_Relative_Boundary_Modeling_CVPR_2023_paper.pdf"},"title":"Benchmarking Data Efficiency and Computational Efficiency of Temporal Action Localization Models","url":"https://arxiv.org/pdf/2308.13082"},"title":"TriDet: Temporal Action Detection with Relative Boundary Modeling - ADS","url":"https://ui.adsabs.harvard.edu/abs/2023arXiv230307347S/abstract"},"title":"TriDet: Temporal Action Detection with Relative Boundary Modeling | IEEE Conference Publication | IEEE Xplore","url":"https://ieeexplore.ieee.org/document/10203543/"]]

Based on the search results, I found relevant information about TriDet's SGP layer and its comparison with ActionFormer.

The SGP layer replaces self-attention with a convolutional-based architecture that includes an instant-level branch and a window-level branch.

When compared with ActionFormer, which adopts a self-attention mechanism in a sliding window with window size 7, the SGP layer achieves a 1.5% improvement in average mAP. Specifically, ActionFormer achieved 66.8% average mAP on THUMOS14, while TriDet with the SGP layer achieved 69.3% average mAP.

The SGP layer addresses the rank loss problem plaguing self-attention mechanisms where inter-snippet feature similarity is high by replacing self-attention with a convolutional-based architecture.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

11. assistant (2026-06-16T03:23:09.116Z)

Both contested elements are now corroborated. Let me verify the ActionFormer 66.8% figure independently against ActionFormer's own paper, since that's a load-bearing baseline number in the claim.

12. user (2026-06-16T03:23:14.995Z)

Web search results for query: "ActionFormer THUMOS14 average mAP 66.8 paper results table"

Links: [{"title":"ActionFormer: Localizing Moments of Actions with Transformers","url":"https://arxiv.org/pdf/2202.07925"},"title":"Benchmarking Data Efficiency and Computational Efficiency","url":"https://cveu.github.io/2023/papers/11.pdf"},"title":"Supplementary Materials for ActionFormer: Localizing Moments of Actions","url":"https://www.ecva.net/papers/eccv_2022/papers_ECCV/papers/136640485-sup.pdf"},"title":"GitHub - csh415/actionformer_performance","url":"https://github.com/csh415/actionformer_performance"},"title":"GitHub - chrisindris/actionformer_release: Code release for ActionFormer (ECCV 2022) · GitHub","url":"https://github.com/chrisindris/actionformer_release"},"title":"GitHub - happyharrycn/actionformer_release: Code release for ActionFormer (ECCV 2022) · GitHub","url":"https://github.com/happyharrycn/actionformer_release"},"title":"actionformer_release/README.md at main · minzhang15/actionformer_release","url":"https://github.com/minzhang15/actionformer_release/blob/main/README.md"},"title":"(PDF) ActionFormer: Localizing Moments of Actions with Transformers","url":"https://www.researchgate.net/publication/358655346_ActionFormer_Localizing_Moments_of_Actions_with_Transformers"},"title":"TBT-Former: Learning Temporal Boundary Distributions for Action Localization","url":"https://arxiv.org/pdf/2512.01298"},"title":"Boundary Discretization and Reliable Classification Network for Temporal Action Detection","url":"https://arxiv.org/pdf/2310.06403"}]

Based on the search results, I found information about ActionFormer's THUMOS14 results:

ActionFormer achieves an average mAP of 66.8% ([0.3 : 0.1 : 0.7]), with an mAP of 71.0% at tIoU=0.5 and an mAP of 43.9% at tIoU=0.7 on the THUMOS14 dataset.

Additionally, the supplementary materials show results by varying feature stride: with stride 4, the model achieves 71.0 mAP at tIoU=0.5, 43.9 mAP at tIoU=0.7, and 66.8 average mAP.

The original paper is titled "ActionFormer: Localizing Moments of Actions with Transformers" and was published at ECCV 2022, with the arxiv version available at <https://arxiv.org/pdf/2202.07925>.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

13. assistant (2026-06-16T03:23:25.835Z)

All elements of the claim are now independently verified. Let me consolidate the adversarial findings before returning my verdict.

The claim has four sub-assertions, and I checked each against primary/credible sources:

1. **TriDet 69.3% average mAP on THUMOS14** ? Verified verbatim from the arXiv abstract (2303.07347) and the supporting quote matches exactly.
2. **ActionFormer at 66.8% as the prior best** ? The abstract itself only says "previous best by 2.5%" without naming it, but: $69.3 - 66.8 = 2.5$ exactly; ActionFormer's own paper (arXiv 2202.07925) independently reports 66.8% average mAP on THUMOS14; and TriDet's ablation explicitly compares SGP vs ActionFormer's sliding-window self-attention. The "ActionFormer at 66.8%" attribution is correct, not an overreach.
3. **74.6% of its latency** ? Verified verbatim from the abstract.
4. **Convolution-only / no self-attention** ? Verified. TriDet's SGP layer explicitly *replaces* self-attention with a convolutional architecture (instant-level + window-level branches) to fix the rank-loss problem of self-attention. This is the central architectural contribution. The earlier WebFetch phrase "TriDet uses self-attention" was a misreading by the summarizer ? the actual mechanism replaces it.

One minor pedantic note: the paper says "2.5%" (the abstract's own wording), and the claim phrases it as "2.5 points," which is the more technically correct framing (mAP is already a percentage, so the gap is percentage points). This is a refinement, not an error.

No contradicting evidence surfaced. The source is the primary CVPR 2023 paper, matching the claim's strength. The claim is current (CVPR 2023, well within scope) and not marketing ? it's a peer-reviewed self-report, corroborated by the independent ActionFormer baseline.

14. user (2026-06-16T03:23:30.978Z)

Structured output provided successfully

15. assistant (2026-06-16T03:23:37.406Z)

Claim verified as accurate ? refuted=false with high confidence.

Summary of the adversarial review: I attempted to refute all four sub-claims and each held up against primary sources.

- The two numeric/latency figures (69.3% avg mAP, 74.6% latency) are quoted verbatim from the TriDet abstract (arXiv 2303.07347 / CVPR 2023).
- The "ActionFormer at 66.8%" baseline ? though the abstract only says "previous best" ? is independently confirmed by ActionFormer's own paper (66.8% avg mAP on THUMOS14) and is arithmetically consistent ($69.3 - 66.8 = 2.5$).
- The "convolution-only, no self-attention" characterization is correct and is in fact the paper's headline contribution: the SGP layer *replaces* self-attention with a convolutional

architecture to address the rank-loss problem. (One intermediate auto-summary mis-stated that TriDet "uses" self-attention; the paper's actual mechanism replaces it, so the claim is right and the summary was wrong.)

Source quality (peer-reviewed CVPR 2023 primary paper + independent baseline confirmation) matches the claim's strength, and nothing contradicts it.